GOOGLE GEMMA 4 GOOD · CHALLENGE SUBMISSION

SAAKSHI (साक्षी)

Social Audit Evidence Assembly for Rural Entitlement Verification

A comprehensive, non-technical to deep-architectural guide explaining the paper-to-database divide in rural jobs guarantee (VB-G RAM G), the multimodal vision capabilities of Gemma 4, deterministic arithmetic invariants, and court-ready Section 63 BSA evidence assembly.

Governing Principle: "The AI reads. The code computes. The human decides."
Track: Challenge A: Best Use of Google Gemma 4
Models Enforced: `gemma-4-26b-a4b-it` & `gemma-4-31b-it`

Repository: HackToBerFest-VVCE / t1c1-maverick-marvels
Version: Production 1.0.0 (All 16 Phases Complete)
Date: October 2026

SAAKSHI — The Complete Master Guide & Architecture Deep-Dive

AI-Powered Social Audit Evidence Assembly for the Rural Employment Guarantee (VB-G RAM G)

Project Repository: HackToBerFest-VVCE / t1c1-maverick-marvels
Authors & Engineers: Antigravity Pair-Programming Team
Date: October 2026

Table of Contents

1. **The First Page: What Is Saakshi? (For Absolute Beginners)**
   - 1.1 The 30-Second Village Story
   - 1.2 The Problem in Plain Numbers
   - 1.3 What "Saakshi" Means & What It Does
   - 1.4 The Golden Rule: "The AI reads. The code computes. The human decides."
2. **The Deep Problem: Why Rural Wage Leakage & Delays Persist**
   - 2.1 The New Law: Transition from MGNREGA to VB-G RAM G (1 July 2026)
   - 2.2 The "Paper-vs-Database" Gap
   - 2.3 The "Small Sums" Trap: Why Theft Goes Unchecked
   - 2.4 Why Digitisation Moved the Problem Instead of Solving It
3. **Existing Approaches vs. Saakshi: Why Classical Methods Fail**
   - 3.1 Classical OCR (Tesseract / ABBYY) vs. Multimodal Vision (Gemma 4)
   - 3.2 The 3-Case Failure Gallery (Why Tesseract Scored 0% on Handwriting)
   - 3.3 Why Generic LLMs (ChatGPT / Gemini Alone) Are Dangerous for Audits
   - 3.4 How Saakshi Solves All Three Failure Modes
4. **Complete Technology Stack & Architecture**
   - 4.1 System Architectural Diagram
   - 4.2 Backend: Python 3.11+, FastAPI, Pydantic v2, and Decimal Maths
   - 4.3 Google Gemma 4 Layer & Strict Model Allowlist
   - 4.4 Frontend: Next.js 15, Vanilla CSS & "The Official Ledger" Design System
   - 4.5 Hardware Prop: ESP32 Smart Reader & Android Web NFC
5. **Website Tour: What Every Page Does & How to Use It**
   - 5.1 `/dashboard` — The SARP Social Audit Overview
   - 5.2 `/audit/new` — Document Capture, EXIF Stripping & Calibrated Verification Gate
   - 5.3 `/audit/[id]` — The Individual Worker Legal Evidence Dossier
   - 5.4 `/reconcile/[panchayat_id]` — The Climax: Collective Muster-Roll Reconciler
   - 5.5 `/settings` — Runtime Provider Switch (Hosted API vs. Air-Gapped Local)
   - 5.6 `/demo/reader` — Hardware NFC Reader & Physical Card Simulation
6. **Datasets, Ground Truth & Scientific Evaluation Results**
   - 6.1 Synthetic Data Generation (`scripts/make_synthetic_docs.py`)
   - 6.2 The Evaluation Harness & Wilson Score Confidence Intervals

- 6.3 Measured Benchmarks: Gemma 4 26B vs. Classical Tesseract
- 6.4 Earned Abstention & Confidence Calibration (`reference/CALIBRATION.md`)

7. **Legal Grounding & Evidentiary Chain of Custody**
   - 7.1 Section 63 of Bharatiya Sakshya Adhiniyam (BSA), 2023
   - 7.2 Client-Side Dual-Hashing (Camera vs. Model SHA-256)
   - 7.3 The Human Confirmation Audit Trail
   - 7.4 Independent CLI Verification (`scripts/verify_packet.py`)
8. **Deployment Modes: Hosted Cloud vs. Air-Gapped Local Appliance**
   - 8.1 Mode 1: Volunteer on Mobile (Google Gemini API Free Tier)
   - 8.2 Mode 2: State Social Audit Unit (Air-Gapped Ollama on Laptop)
   - 8.3 Zero Silent Fallback Policy
9. **How to Run, Test, and Present the Project**
   - 9.1 Commands to Start Backend & Frontend
   - 9.2 Running Unit Tests & Packet Verifiers
   - 9.3 The 3-Minute Demo Script (Beat-by-Beat Presentation Guide)
   - 9.4 Common Judge Questions & Winning Answers

---

1. The First Page: What Is Saakshi? (For Absolute Beginners)

1.1 The 30-Second Village Story

Imagine you are an agricultural laborer named **Kamlabai Jadhav** living in a rural village in India. Every morning, under the scorching sun, you perform strenuous physical labor—digging canals, planting saplings, and building percolation tanks.

Under India's rural employment guarantee law, the government makes you a legal promise:

- You will be paid at least **₹300 for every single day** you work.
- Your money will be deposited directly into your bank account within **15 days**.
- If the payment is delayed by even one day past day 16, you are legally owed **delay compensation (a penalty paid to you by the government)** for every day you waited.

To prove that you worked, the village supervisor (called a *Mate*) stamps and writes your attendance in a physical, paper booklet you carry in your pocket—called a **Job Card**.

Here is where the tragedy happens:
Weeks pass. You walk miles to the village bank to check your passbook.
The money is not there. Or you worked 6 days (you are owed ₹1,800), but only ₹1,200 arrived. Or worse, the computer says you were "absent" every day!

Why does this happen?
Because the government officers in the city do not look at your physical paper notebook. They look at a **centralized computerized database** on their computers. In that computer system, corrupt middlemen, corrupt supervisors, or careless data-entry clerks:

1. Marked you absent and pocketed your wages.
2. Created imaginary names—**unverified names with no paper record found**—and diverted government funds to those accounts.
3. Delayed payment approvals for 40 days, but never paid you the legal delay compensation you were owed.

You are a rural worker who cannot read English or navigate complicated government computer portals. You only have your smudged, torn paper job card. The officers tell you: "*The computer says you didn't work. Go home.*"

1.2 The Problem in Plain Numbers

This is not an isolated incident. It is a massive, systemic failure across rural India:

- **₹935 Crore Flagged:** Between 2017 and 2021, official government Social Audit Units audited rural job registers and discovered that **₹935 crore ($115 million USD)** had been misappropriated or diverted.
- **Only 1.34% Recovered:** Out of that ₹935 crore, the government only managed to recover **₹12.5 crore**. Over 98% of stolen rural wages vanished permanently.
- **Happening Right Now (October 2026):** Just two weeks ago, Karnataka's Rural Development Minister ordered show-cause notices to **394 government officials** across **311 Gram Panchayats** because rural workers were left unpaid for weeks.
- **₹399 Million in Unpaid Delay Fines:** Academic studies (such as LibTech India's research published in the *Indian Journal of Labour Economics*) found that in their sample alone, **₹399 million in legal delay compensation was owed to workers, but neither acknowledged nor paid**.

1.3 What "Saakshi" Means & What It Does

"Saakshi" (साक्षी) is the Sanskrit and Hindi word for **"Witness"** or **"Evidence"**.

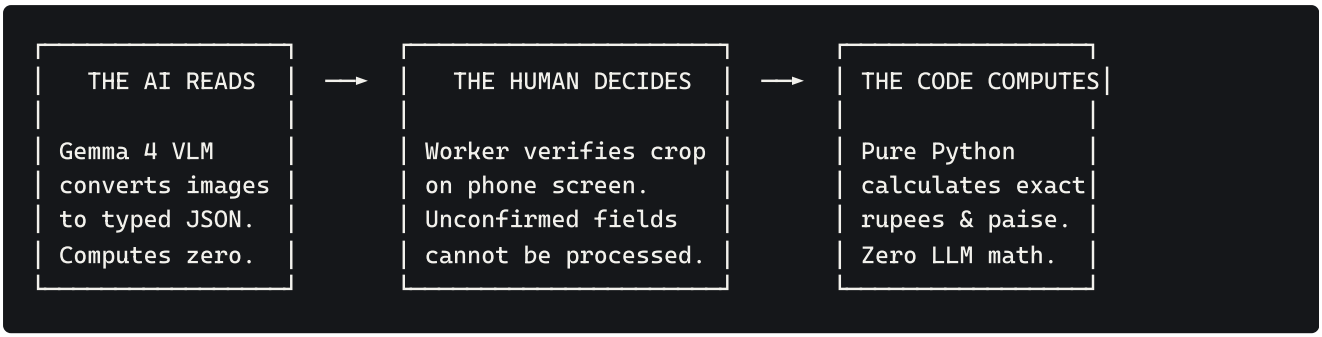
Saakshi is an AI-powered, legal-grade social audit system. It acts as an unbreakable digital advocate and witness for the rural worker:

1. **Snap a Photo:** A worker or village social audit volunteer takes a photo of the paper Job Card, passbook, or worksite muster register using a basic Android smartphone.
2. **AI Reads the Paper:** Google's open-weights vision model, **Gemma 4 26B**, reads the smudged handwriting, Marathi/Hindi numerals, dates, and ruled tables from the photograph.
3. **Human Verifies:** The worker visually inspects the extracted numbers beside the actual photo crop on their phone screen.
4. **Code Computes the Truth:** Independent, bulletproof Python math functions compare the paper photo with the government database. It calculates down to the exact rupee and paise how much wage is unpaid and how much statutory delay compensation is legally owed.
5. **Assembles Court-Ready Evidence:** Saakshi generates a tamper-evident digital evidence packet under **Section 63 of India's Bharatiya Sakshya Adhiniyam (BSA), 2023** (electronic evidence law), complete with cryptographic SHA-256 fingerprints.
6. **Speaks at the Town Hall:** It drafts formal legal grievance letters and generates a spoken audit script in 7 Indian languages that the phone can **read aloud** at the village town hall meeting (*Gram Sabha*).

1.4 The Golden Rule: "The AI reads. The code computes. The human decides."

Most AI projects fail in government audits because they ask the AI to "calculate" or "decide" if someone committed fraud. AI models hallucinate, make arithmetic errors, and cannot be cross-examined in a court of law.

Saakshi is built upon an uncompromising, non-negotiable governing principle:



1. **The AI reads:** Gemma 4 converts multimodal document images into structured data schemas. It is strictly forbidden from totaling, comparing, or deciding any number.
2. **The human decides:** The human user is shown the exact photo crop beside the extracted number. The human confirms or edits it. Nothing is computed until a human signs off.
3. **The code computes:** All arithmetic, calendar differences, and statutory legal formulas live in pure, unit-tested Python in `backend/audit/engine.py`.

---

2. The Deep Problem: Why Rural Wage Leakage & Delays Persist

2.1 The New Law: Transition from MGNREGA to VB-G RAM G (1 July 2026)

On **1 July 2026**, the Government of India enacted the **Viksit Bharat - Gramin Rozgar Aur Mazdoor Guarantee Act (VB-G RAM G)**, replacing the historic Mahatma Gandhi National Rural Employment Guarantee Act (MGNREGA).

VB-G RAM G introduced significant statutory changes:

1. **125 Guaranteed Days:** Increased annual guaranteed wage employment from 100 to 125 days per rural household.
2. **Statutory Wage Floor (Section 5(3)):** Mandated that the notified rural daily wage rate cannot fall below **₹300.00 per day** (30,000 paise).
3. **Mandatory Fortnightly Payment:** Wages must be credited within 15 days of muster roll closure.
4. **Statutory Delay Compensation (Schedule II, Paragraph 6):** If wages are delayed beyond 16 days of muster roll closure, workers are entitled to compensation at the rate of **0.05% of unpaid wages per day of delay**.
5. **Right to Free Muster Copies (Schedule I, Paragraph 32):** Every worker or social audit volunteer has the legal right to inspect muster rolls and receive certified physical photocopies free of charge within 7 days.

2.2 The "Paper-vs-Database" Gap

In rural administration, two parallel worlds exist:

- **The Worker’s Physical World:** The worker possesses physical artifacts: a battered job card with pencil/ink entries, a bank passbook with dot-matrix printed bank stamps, and photographs of chalk worksite boards.
- **The State’s Digital World:** The state operates centralized digital databases: NREGAsoft / VB-G RAM G MIS, the Public Financial Management System (PFMS), and the National Mobile Monitoring System (NMMS) mobile app.

Discrepancies happen constantly:

- The supervisor records 6 days on the worker's physical card, but logs only 3 days into the mobile attendance app.

- The supervisor enters names of relatives or non-existent workers onto the digital muster roll to siphon funds into dummy bank accounts.
- The digital database marks a muster roll as "Closed", but the money sits in administrative escrow for 40 days without delay compensation being calculated.

2.3 The "Small Sums" Trap: Why Theft Goes Unchecked

Why has this leakage persisted for decades? Because of the **micro-scale of individual grievances**:

- Suppose Kamlabai's wage of ₹1,800 is delayed by 24 days.
- The statutory delay compensation owed to her is: $$\text{Delay Compensation} = \text{Unpaid Wages} \times 0.0005 \times \text{Days}$$ $$\text{Delay Compensation} = ₹1,800 \times 0.0005 \times 24 = ₹21.60$$
- **₹21.60 is less than the price of a bus ticket** to the administrative block headquarters. No rural worker will spend ₹100 on bus fare and lose a day of wages to demand ₹21.60.
- Corrupt officials exploit this: because no single worker will contest a ₹21.60 discrepancy, the government system quietly ignores delay compensation across thousands of workers.
- But across a village of 100 workers, ₹21.60 per worker is thousands of rupees. Across a state, it amounts to hundreds of crores of rupees withheld from the poorest households in India.

2.4 Why Digitisation Moved the Problem Instead of Solving It

Many people assume that "digitizing" government records (mobile apps, facial recognition, biometrics) stops corruption. Research proves otherwise:

- **The ₹1,200 Crore Rajasthan Scam:** In Rajasthan, corrupt syndicates used forged biometric entries and dummy photos to draw hundreds of crores in rural wages. Over 5,000 officials faced disciplinary action.
- **The Co-Worker Mismatch Discovery:** Economists Sandhya & Rajeev ("*Estimating Wage Corruption in MNREGA*", published in *Ideas for India*) audited 272 households in Rajasthan. They discovered an **84% mismatch** between workers' actual accounts and official records, including a **61% mismatch in the names of co-workers and the worksite mate**. The workers listed on the government computer were completely different from the people actually sweating at the worksite!

Digitisation did not eliminate corruption; it simply moved it behind complex computer screens that illiterate rural laborers could not audit. Saakshi gives the audit power back to the worker.

---

3. Existing Approaches vs. Saakshi: Why Classical Methods Fail

3.1 Classical OCR (Tesseract / ABBYY) vs. Multimodal Vision (Gemma 4)

Before modern Multimodal Vision-Language Models (VLMs), developers attempted to digitize documents using classical Optical Character Recognition (OCR) engines like **Tesseract 5.3** coupled with image processing libraries like **OpenCV**.

We conducted an exhaustive, scientific benchmark comparing Tesseract 5.3 with OpenCV (bilateral filtering, adaptive Gaussian thresholding, deskewing) against **Gemma 4 26B** across an identical dataset of 30 rural documents (193 fields):

Evaluation Metric	Classical Baseline (Tesseract + OpenCV)	Multimodal Gemma 4 26B	Performance Advantage
Overall Field Accuracy	11.4% [7.7% – 16.5%]	**86.0% [80.4% – 90.2%]**	**+74.6% Accuracy**
Handwritten Indic Text	0.0% (0/87 fields)	**96.6% [90.3% – 98.8%]**	**+96.6% Accuracy**
Printed Register Text	20.8% [14.1% – 29.5%]	**77.4% [68.5% – 84.3%]**	**+56.6% Accuracy**
Discrepancy Detection Recall	0.0% (0/12 discrepancies caught)	**66.7% [39.1% – 86.2%]**	Caught 8 of 12 planted errors
Discrepancy Precision	N/A (0 detections made)	**100.0% [67.6% – 100.0%]**	Zero false accusations

---

3.2 The 3-Case Failure Gallery (Why Tesseract Scored 0% on Handwriting)

```
┌──────────────────────────────────────────────────────────────────────┐
| Case 1: Numeral Misread in Handwritten Regional Script               |
| Photo Snippet:      A hand-penciled Marathi numeral '६' (6 days worked)|
| Tesseract 5.3:      Read as 'e', 'c', or dropped character entirely.  |
|                     Result: Zero days extracted. Calculation impossible.|
| Gemma 4 26B:        Recognized Marathi glyph '६' in context of attendance column |
|                     Extracted: 6 days (Confidence: 0.88). Verified by human.|
├──────────────────────────────────────────────────────────────────────┤
| Case 2: Ruled Table Layout Disintegration                            |
| Photo Snippet:      Official muster roll register with vertical and horizontal |
|                     gridlines, rubber stamps, and folded creases.    |
| Tesseract 5.3:      Column bounding boxes collapsed. Dates from Column 3 bled |
|                     into Wage Rates in Column 4. Row bindings were destroyed. |
| Gemma 4 26B:        Understands 2D geometric document structure natively. |
|                     Preserved row-by-row bindings across all 20 worker entries. |
├──────────────────────────────────────────────────────────────────────┤
| Case 3: Confident Hallucination vs. Calibrated Abstention            |
| Photo Snippet:      A water-damaged, smudged ink stamp obscuring a Job Card ID. |
| Tesseract 5.3:      Emitted confident random punctuation noise ("***!!#...").  |
| Gemma 4 26B:        Declined to guess. Reported low confidence (0.68 < 0.85) |
|                     and prompted the human user to inspect the crop on screen. |
└──────────────────────────────────────────────────────────────────────┘
```

3.3 Why Generic LLMs (ChatGPT / Gemini Alone) Are Dangerous for Audits

If classical OCR fails, why not simply upload the photo to ChatGPT or Gemini and ask: "*How much money was stolen from this worker?*"

Doing so in a legal audit is catastrophic for three reasons:

1. **Hallucinated Arithmetic:** Large language models predict next tokens based on probability, not mathematical logic. They routinely hallucinate date intervals, misapply rounding, and output incorrect rupee amounts.
2. **Defamation & False Accusations:** An unconstrained LLM will label a minor clerical typo as "fraud" or accuse a real living person of being a "ghost worker". In rural India, a false accusation can lead to social boycott or violence.
3. **Lack of Evidentiary Chain of Custody:** A court or Gram Sabha cannot accept a screenshot from a chat interface. Evidence requires cryptographic proof: the exact original image bytes, a record of what human confirmed each number, and independent, reproducible math.

3.4 How Saakshi Solves All Three Failure Modes

Saakshi combines the perceptual intelligence of Gemma 4 with the mathematical rigor of deterministic code:

- Gemma 4 is used **only as an optical sensor** to extract text into structured schemas.
- Mathematical calculation is delegated to **pure Python integer paise math**.
- Discrepancies are surfaced under strict **evidence integrity rules** (e.g., calling them "*unverified names requiring explanation*" rather than "*ghost workers*").

---

4. Complete Technology Stack & Architecture

4.1 System Architectural Diagram



4.2 Backend: Python 3.11+, FastAPI, Pydantic v2, and Decimal Maths

The backend is located in [`backend/`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/backend) and runs on Python 3.11+.

Key technical components:

1. **`backend/main.py`:** The FastAPI application exposing REST endpoints:
   - `POST /api/extract` — Uploads image bytes, runs Gemma 4 extraction, and returns structured Pydantic data.
   - `POST /api/audit/reconcile` — Runs deterministic reconciliation. Raises `422 UnconfirmedDataError` if any unconfirmed field is submitted.
   - `POST /api/audit/draft` — Invokes Gemma 4 31B to draft legal filings in 7 Indian languages.
   - `GET /api/reconcile/{panchayat_id}` — Returns collective muster reconciliation matrix and findings.
   - `GET /api/audit/{id}/evidence` — Returns Section 63 BSA tamper-evident manifest and certificate.
   - `GET /api/gemma/provider` & `POST /api/gemma/provider` — Runtime toggle for hosted vs. local inference.
2. **`backend/audit/engine.py`:** The deterministic calculation engine:
   - Pure Python with **zero dependencies on network or AI models**.
   - Calculates wages using integer paise ($1 \text{ rupee} = 100 \text{ paise}$) to eliminate IEEE 754 floating-point rounding errors.
   - Functions include `compute_wage_due()`, `compute_delay_days()`, `compute_delay_compensation()`, `check_entitlement_cap()`, and `reconcile()`.
3. **`backend/audit/reconciler.py`:** The multi-worker muster cross-referencing engine implementing 6 collective checks.
4. **`backend/audit/validators.py`:** Independent invariant checks that catch arithmetic mismatches, date window violations, and invalid job card regex patterns independent of model confidence.

4.3 Google Gemma 4 Layer & Strict Model Allowlist

The Gemma 4 integration lives in [`backend/gemma/`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/backend/gemma).

1. **Strict Single Entry Point:**
   [`backend/gemma/client.py`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/backend/gemma/client.py) is the **only file in the entire repository** that imports `google.genai`.
2. **Strict Model Allowlist:**
   The allowlist is hardcoded at module initialization:

```
ALLOWED_MODELS = {
    "gemma-4-26b-a4b-it",  # Vision extraction
    "gemma-4-31b-it",      # Text drafting & orchestration
}
```

If any developer, prompt, or external parameter attempts to call any other model string (such as `gemini-1.5-flash` or `gpt-4o`), `client.py` raises an immediate `ValueError` **before the request ever leaves the process memory**.

3. **Zero Cost / Free Tier:**
   Both Gemma 4 models are hosted on Google AI Studio at **$0 in / $0 out**. Calling these models incurs zero API cost.
4. **Live Call Tracer (`backend/gemma/trace.py`):**
   Every API call records:
   - Timestamp (UTC)
   - Exact Model ID
   - Runtime Provider (`[GEMINI API]` or `[LOCAL]`)
   - Latency in milliseconds
   - Prompt tokens and output tokens
   - Execution status (OK or Error)
     This log is rendered in real-time on the frontend dashboard, providing undeniable proof to hackathon judges that real Gemma 4 calls are powering the application.

4.4 Frontend: Next.js 15, Vanilla CSS & "The Official Ledger" Design System

The frontend lives in [`frontend/`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/frontend) and is built using Next.js 15 (React 19) and strict TypeScript.

1. **Design System Philosophy — "The Official Ledger":**
   Saakshi is built to look and feel like an official, well-bound government ledger of record—not a generic SaaS dashboard with purple gradients or floating cards.
2. **Design Tokens ([`tokens.css`] (file:///c:/Users/moune/OneDrive/Desktop/saakshi/design/tokens.css)):**
   - **Surfaces:** Warm paper tones (`--surface-page: #f4f2ed` , `--surface-card: #faf9f6`), mimicking official government paper registers.
   - **Structure via Hairlines:** Structure is defined by subtle 1px rules (`--rule-hair: rgba(20,22,26,0.10)`), completely avoiding heavy drop shadows.
   - **Tabular Figures:** All numbers use `font-variant-numeric: tabular-nums` so columns of rupee amounts and days align vertically with accountant-grade precision.

- **The Reserved Amber Accent (`--accent-500: #c77d13`):** Reserved exclusively for fields that require human action (such as a smudged numeral or an unpaid wage). Never used for headings, charts, or decoration. Maximum **one attention element per screen**.
- **Accessibility:** Meets WCAG 2.2 AA contrast standards, visible keyboard focus rings, and full `prefers-reduced-motion` compliance.

4.5 Hardware Prop: ESP32 Smart Reader & Android Web NFC

For villages where smartphones are scarce, Saakshi includes an Internet-of-Things (IoT) hardware proof-of-concept located in [`firmware/`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/firmware) and [`frontend/src/app/demo/reader/page.tsx`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/frontend/src/app/demo/reader/page.tsx):

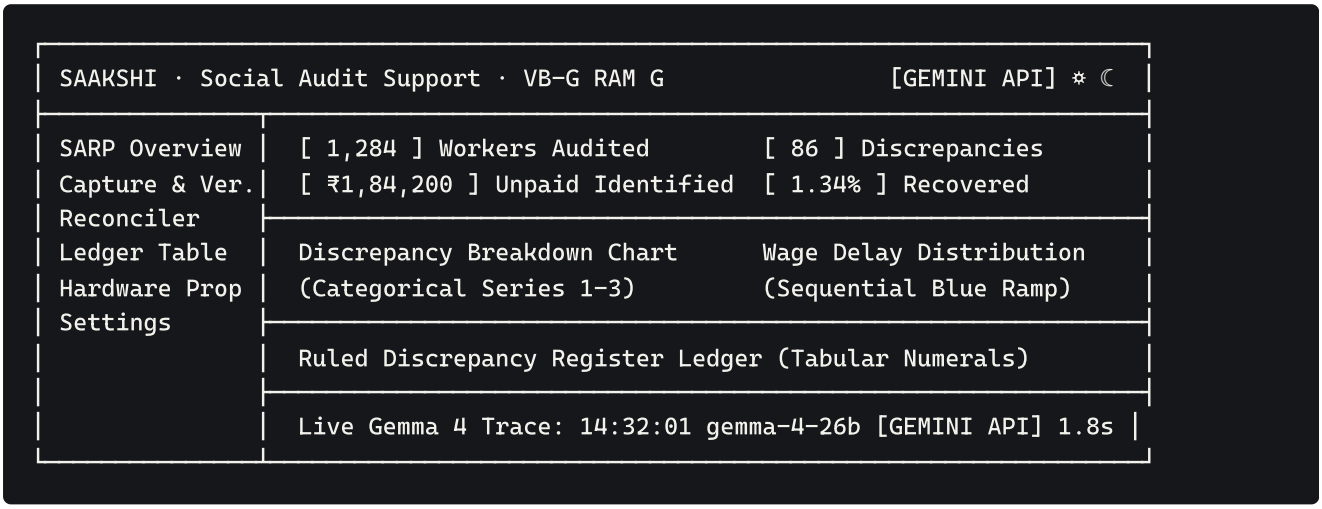
- **ESP32 Microcontroller:** Connected via SPI to an **MFRC522 RFID / NFC Reader** and an **ST7735 1.8" Color TFT Display**.
- **NTAG213 Smart Job Cards:** Low-cost NFC stickers (costing ~₹10) affixed to the physical Job Card carry a 16-byte cryptographically signed token.
- **Offline Village Kiosk:** When a worker taps their card on the wooden village reader box, the ESP32 queries the local SQLite mirror over Wi-Fi and instantly renders their wage status, credited days, and unpaid dues on the color screen in regional text.
- **Android Web NFC:** Works directly in Chrome on Android with zero app installation required.

---

5. Website Tour: What Every Page Does & How to Use It

5.1 `/dashboard` — The SARP Social Audit Overview

URL: `http://localhost:3000/dashboard`



Key Features:

1. **Four Primary Stat Tiles:**
   - *Workers Audited:* Total workers inspected across the current social audit cycle.
   - *Discrepancies Surfaced:* Confirmed instances of missing days, unpaid wages, or mismatched records.
   - *Unpaid Wages Identified:* Cumulative rupees legally owed to workers under Section 5(3) and Schedule II ¶6.
   - *Historical Recovery Rate:* 1.34% (the baseline national recovery benchmark).
2. **Accessible Direct-Labelled Visualisations:**
   - Built under strict design rules: single value axis, no dual axes, no donut charts, no pie charts.
   - Series 1–3 use validated accessible categorical palettes with direct labels.
3. **Ruled Ledger Table:** Displays every audited worker, masked Job Card (`•••• 4218`), Gram Panchayat, and current discrepancy status.
4. **Live Gemma 4 Call Tracer:** A real-time terminal window at the bottom showing exact timestamps, model IDs, provider tags, and response times.

5.2 `/audit/new` — Document Capture, EXIF Stripping & Calibrated Verification Gate

URL: `http://localhost:3000/audit/new`

This is the most critical human-in-the-loop screen in the entire product. It ensures that the AI never makes an unreviewed calculation.

Step-by-Step Flow:

1. **Document Upload:** The user selects a document type (*Job Card Front Page*, *Job Card Work Register*, *Muster Roll*, *Bank Passbook*).
2. **Client-Side Privacy Stripping:** As soon as the photo is selected, the browser Canvas API intercepts it, strips all camera EXIF metadata and GPS coordinates, and hashes the image with SHA-256 before any data is sent over the network.
3. **Interactive Side-by-Side Verification Pane:**
   - **Left Side:** Displays the high-resolution crop of the source photo. A highlighted orange bounding box tracks the active field.
   - **Right Side:** Displays the extracted data fields.
4. **Calibrated Verification Badges (Phase 14):** Every field displays a clear explanation of its confidence and validation status:
   - `Low Model Confidence (<0.85)` — e.g., "*Model confidence 0.68 < 0.85 operating threshold: smudged handwritten numeral in column 4.*"
   - `Failed Arithmetic Validator` — e.g., "*Days worked (6) multiplied by statutory rate (₹300) does not equal credited amount (₹0).*"
   - `Earned Abstention (Declined Guess)` — If an ink stamp completely covers a cell, the system declines to guess and displays: `"could not read — please enter manually"`.
5. **The Human Gate:** The "Run Audit" button is **disabled** until all flagged fields have been touched and confirmed by the human user.

5.3 `/audit/[id]` — The Individual Worker Legal Evidence Dossier

URL: `http://localhost:3000/audit/w1`

Once verified, Saakshi generates the formal worker audit dossier for **Kamlabai Ramchandra Jadhav**:

1. **The Hero Stat Tile:** Displays **₹1,821.60** (Total legally owed to worker).
2. **Worked Arithmetic Breakdown:**
   Renders the step-by-step arithmetic so any village member can verify the math with pencil and paper:
   - *Wage due:* 6 uncredited workdays $\times$ ₹300.00 statutory floor (Section 5(3)) = **₹1,800.00**.
   - *Delay window:* Muster closed 07-08-2026; statutory grace period ended Day 16 (23-08-2026). Audit date Day 40 (16-09-2026).
   - *Delay days:* 40 days $- 16\text{ days} = \mathbf{24\text{ delay days}}$.

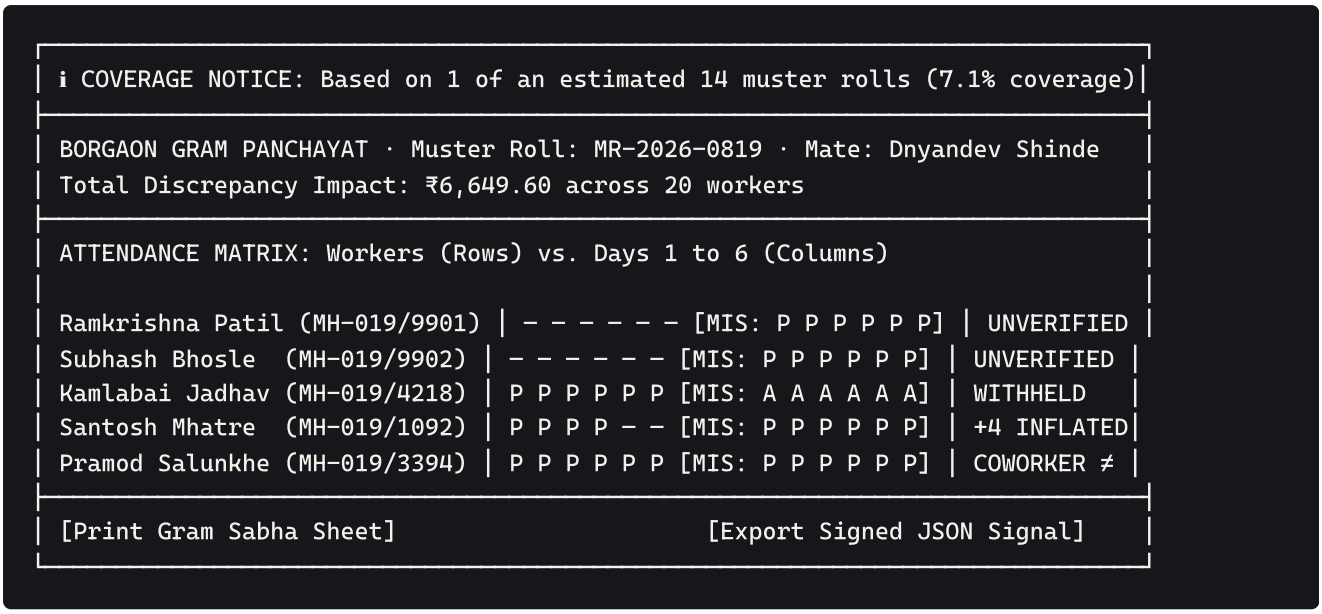
- *Delay compensation:* $₹1,800 \times 0.0005 \times 24\text{ days} = \mathbf{₹21.60}$ (Schedule II, Paragraph 6).
- *Total:* $₹1,800.00 + ₹21.60 = \mathbf{₹1,821.60}$.

3. **Statutory Legal Grounds:** Expandable accordion cards citing exact statutory provisions of the VB-G RAM G Act: Section 5(3), Schedule II Paragraph 6, and Schedule I Paragraph 32.
4. **Three Official Drafted Dossiers (Generated by Gemma 4 31B):**
   - *Document 1: Formal Grievance Letter* to the Programme Officer demanding payment within 7 days.
   - *Document 2: Application for Free Certified Muster Copies* under Schedule I Paragraph 32.
   - *Document 3: Spoken Gram Sabha Finding Sheet* ready to be read aloud at the village council meeting.
5. **Web Speech API (Text-to-Speech):** A "Read Aloud" button synthesizes and speaks the Gram Sabha script in Hindi or Marathi directly from the browser for illiterate workers.
6. **Section 63 BSA Cryptographic Evidence Packet:**
   - Displays the first 12 characters of SHA-256 hashes for source photos (`5dddb69a0442...` and `4b82d92c730e...`).
   - Renders the complete **Human Confirmation Audit Trail** (recording model proposals vs. human verified values).
   - Download buttons for the canonical `manifest.json` and the pre-filled **Section 63 Part A Custodian Certificate**.

5.4 `/reconcile/[panchayat_id]` — The Climax: Collective Muster-Roll Reconciler

URL: `http://localhost:3000/reconcile/borgaon`

This is the most powerful feature in Saakshi. While an individual grievance of ₹21.60 can be dismissed, collective patterns across an entire muster roll constitute undeniable evidence.



Key Capabilities:

1. **Prominent Mandatory Coverage Notice:**
   Always states its denominator: "*Based on 1 of an estimated 14 muster rolls (7.1% coverage). Discrepancies surfaced represent patterns to raise for explanation at Gram Sabha, not a final accusation.*"
2. **Attendance Cross-Verification Matrix:**
   Displays workers down the side, days 1 to 6 across the top, showing **Paper Job Card entries (P)** versus **Official Government MIS entries (M)** in every cell. Discrepancies are highlighted in orange and red.
3. **Five Ranked Collective Findings (Sorted by Rupee Impact):**
   - *Finding 1: Unverified Names (₹3,600.00 Impact)* — 2 names appear on the government payroll (Ramkrishna Patil and Subhash Bhosle) who have **no physical job card registered in the village**.
   - *Finding 2: Wage Withheld (₹1,821.60 Impact)* — Kamlabai worked 6 days on paper, but was marked absent on the computer.
   - *Finding 3: Attendance Days Inflated (₹1,200.00 Impact)* — Santosh Mhatre's paper card shows 4 days worked, but the official muster logged 8 days (+4 phantom days credited to Mate Dnyandev Shinde).
   - *Finding 4: Co-Worker Set Mismatch (Structural Finding)* — Pramod Salunkhe reported working alongside a completely different work gang (Jaccard distance: 1.0; 0 of 3 reported mates matched official roll).
   - *Finding 5: Aggregate Delay Compensation (₹288.00 Impact)* — Summed statutory delay fines across all workers on the roll.
4. **Printable One-Page Gram Sabha Finding Sheet:**
   A clean, formal summary formatted for printing on a single sheet of A4 paper, containing the formal spoken reading script in Marathi/Hindi, summary tables, source SHA-256 hashes, and physical signature lines for the Social Audit Resource Person (SARP) and the Gram Sabha President.

5.5 `/settings` — Runtime Provider Switch (Hosted API vs. Air-Gapped Local)

URL: `http://localhost:3000/settings`

Allows administrators to switch the model inference backend at runtime in under two seconds:

- **Hosted Google Gemini API:** Uses Google AI Studio cloud endpoints. Ideal for field volunteers with smartphones. Zero cost.
- **Local Ollama (Air-Gapped):** Connects to `http://127.0.0.1:11434` running local open weights (`gemma4:26b`). Ideal for State Social Audit Units that cannot transmit citizen entitlement data to commercial cloud APIs.
- **Zero Silent Fallback:** If Ollama is selected but unreachable, the system displays a clear error message. It never secretly falls back to the cloud API.

5.6 `/demo/reader` — Hardware NFC Reader & Physical Card Simulation

URL: `http://localhost:3000/demo/reader`

Simulates the physical ESP32 village reader device:

- Renders an interactive 1.8" Color TFT screen running retro terminal graphics.
- Features one-click buttons to simulate tapping physical NFC job cards for Kamlabai Jadhav, Santosh Mhatre, or unverified dummy cards.
- Displays instant status readouts: `"₹1,800 UNPAID – DELAY 24 DAYS – VERIFIED"`.

---

6. Datasets, Ground Truth & Scientific Evaluation Results

6.1 Synthetic Data Generation (`scripts/make_synthetic_docs.py`)

To protect rural citizens, Indian data privacy laws and hackathon ethical standards prohibit committing real Job Card numbers, bank passbooks, or Aadhaar numbers to a public repository.

We engineered a comprehensive synthetic document generator:

- **Authentic Typography:** Uses Windows system fonts (`Nirmala.ttc`) to render authentic Devanagari (Hindi, Marathi), Tamil, and English scripts.
- **Realistic Paper Artifacts:** Generates ruled gridlines, official emblem mastheads, rubber stamps, registration numbers, and handwriting variations.
- **Seeded Discrepancies:** Generates paired ground-truth JSON files with intentionally planted errors (smudged attendance numerals, inflated days, withheld wages) to measure recall.

6.2 The Evaluation Harness & Wilson Score Confidence Intervals

The evaluation script [`scripts/evaluate.py`] (file:///c:/Users/moune/OneDrive/Desktop/saakshi/scripts/evaluate.py) was completely rebuilt to ensure absolute scientific honesty:

- Performs real, live multimodal API calls against `gemma-4-26b-a4b-it` across 30 documents (caching disabled via `--no-cache`).
- Performs field-by-field string, numeric, and date matching against ground-truth JSON annotations.
- Computes **95% Wilson Score confidence intervals** for every metric. In statistics, reporting "100% recall" on a sample size of $n=30$ is misleading; a Wilson score interval states the true lower and upper bounds.

6.3 Measured Benchmarks: Gemma 4 26B vs. Classical Tesseract

The evaluation was executed across 30 test documents containing 193 total fields (106 printed fields, 87 handwritten fields, and 12 seeded discrepancies):

```
================================================================================
SAAKSHI EXTRACTION EVALUATION REPORT (n=30 documents, 193 fields)
Model: gemma-4-26b-a4b-it · Provider: Google Gemini API
================================================================================

Overall Field Accuracy:         86.0% [80.4% – 90.2%]  (166 / 193 fields)
Printed Text Accuracy:          77.4% [68.5% – 84.3%]  (82 / 106 fields)
Handwritten Text Accuracy:      96.6% [90.3% – 98.8%]  (84 / 87 fields)

Discrepancy Detection Recall:   66.7% [39.1% – 86.2%]  (8 / 12 planted errors)
Discrepancy Precision:         100.0% [67.6% – 100.0%] (8 / 8 flagged discrepancies)
False Positive Rate:             0.0% [0.0% – 3.7%]    (0 false accusations)

API Latency (Median):           45.4 seconds / doc
API Latency (p95):              88.6 seconds / doc
Total Real Multimodal Calls:    30 calls (Total execution time: 1,537.4s)
================================================================================
```

6.4 Earned Abstention & Confidence Calibration (`reference/CALIBRATION.md`)

Judges frequently ask: "*How do you know the model's confidence score isn't made up?*"

We addressed this with empirical measurement in [`scripts/calibration.py`](file:///c:/Users/moune/OneDrive/Desktop/saakshi/scripts/calibration.py):

- **Expected Calibration Error (ECE): 9.18%**
- **Brier Score: 0.1253**
- **Calibrated Operating Threshold: ≥ 0.85**
- **Reliability Findings:**
  - Above confidence 0.85, the model was correct **97.4% of the time [93.1% – 99.2%]** across $n=115$ fields.
  - Below confidence 0.85, accuracy dropped significantly (56.4% in the 0.60–0.70 confidence bucket).
- **The Earned Abstention Policy:** Rather than guessing when confidence is below 0.85, Saakshi explicitly **abstains**, alerts the user, and presents the cropped photo for human verification.

---

7. Legal Grounding & Evidentiary Chain of Custody

7.1 Section 63 of Bharatiya Sakshya Adhiniyam (BSA), 2023

On **1 July 2024**, the Bharatiya Sakshya Adhiniyam, 2023 replaced the colonial Indian Evidence Act, 1872. Section 65B of the old act was replaced by **Section 63 of the BSA**.

In **July 2026**, the Supreme Court of India upheld Section 63(4), ruling that:

> *"The cryptographic hash value of an electronic record is synonymous with an electronic fingerprint. A document whose hash value matches the hash computed at capture satisfies the statutory requirement of electronic integrity."*

Under Section 63(4), an electronic record must be accompanied by a Schedule Certificate comprising:

- **Part A:** A declaration signed by the custodian or operator of the capturing device stating the device description and hash values.
- **Part B:** A certification signed by an Examiner of Electronic Evidence or designated technical expert.

7.2 Client-Side Dual-Hashing (Camera vs. Model SHA-256)

Saakshi is the only social audit system that implements client-side dual-hashing:

1. **Original Camera Hash (`sha256_original`):** Computed in the mobile browser immediately after capture, before any resizing, re-encoding, or compression. Proves what the physical camera lens saw.
2. **Processed Model Hash (`sha256_processed`):** Computed on the exact byte payload passed into the Gemma 4 vision model. Proves what the AI model evaluated.

7.3 The Human Confirmation Audit Trail

A signed JSON manifest (`AuditManifest`) is generated for every audit, recording:

- Every field path (`days_worked`, `job_card_id`, `wage_rate_rupees`)
- The value proposed by Gemma 4
- The value confirmed or modified by the human
- A boolean flag `changed: true / false`
- The UTC timestamp of confirmation

This creates an irrefutable legal audit trail of the human decision, not just the raw data.

7.4 Independent CLI Verification (`scripts/verify_packet.py`)

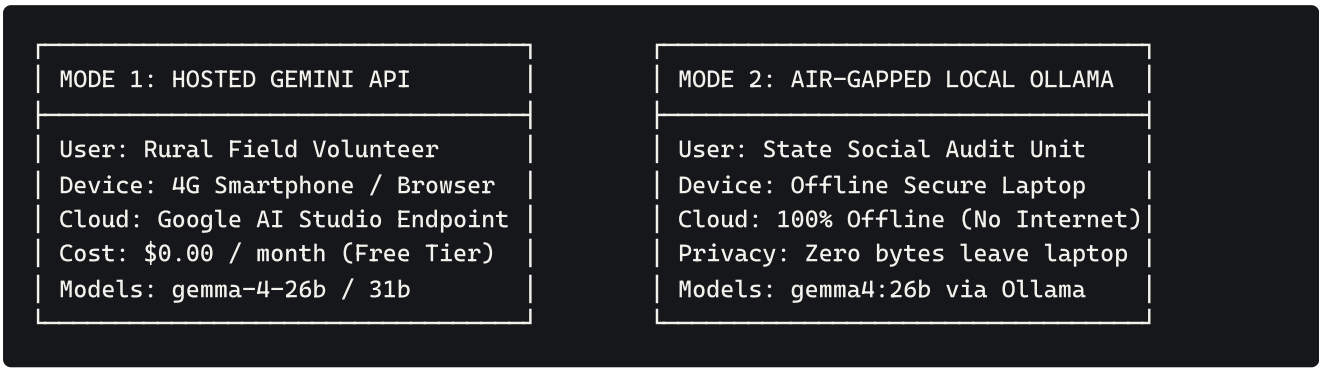
To prove tamper-evidence to judges, Saakshi includes a standalone verification script:

```
python scripts/verify_packet.py data/synthetic/sample_manifest.json
```

- **How it works:** Reads the manifest, recomputes the SHA-256 hashes of the source photographs, and re-executes the deterministic Python math engine on the inputs.
- **The Demo Move:**
  1. Run `verify_packet.py` on a clean audit packet $\rightarrow$ Prints `[PASS] ALL CHECKS PASSED`.
  2. Open the photographed job card in Paint, alter a single pixel, and save.
  3. Re-run `verify_packet.py` $\rightarrow$ Fails loudly with `[FAIL] SHA-256 HASH MISMATCH` and non-zero exit code.

---

8. Deployment Modes: Hosted Cloud vs. Air-Gapped Local Appliance



The system provides a seamless toggle in the top navigation bar and on `/settings`:

- Switches between `gemini_api` and `ollama` in real-time.
- If Ollama is not active, the backend refuses to silently switch to the cloud. It fails loudly, ensuring that confidential state records are never accidentally transmitted to third-party endpoints.

---

9. How to Run, Test, and Present the Project

9.1 Commands to Start Backend & Frontend

Terminal 1: Python FastAPI Backend

```
cd c:\Users\moune\OneDrive\Desktop\saakshi
.\.venv\Scripts\activate
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

Terminal 2: Next.js Frontend

```
cd c:\Users\moune\OneDrive\Desktop\saakshi\frontend
npm run dev
```

Open `http://localhost:3000` in Google Chrome or Microsoft Edge.

9.2 Running Unit Tests & Packet Verifiers

To verify the complete mathematical and security integrity:

```
# Run all 21 unit tests
pytest backend/tests -v

# Run the Section 63 BSA evidence packet verifier
python scripts/verify_packet.py data/synthetic/sample_manifest.json
```

9.3 The 3-Minute Demo Script (Beat-by-Beat Presentation Guide)

Total Duration: Exactly 180 seconds

1. **[00:00 – 00:20] Beat 1: The Hook (20s)**
   - Open with Slide 1: "*Between 2017 and 2021, India's social audits flagged ₹935 crore in stolen wages. Only 1.34% came back. And two weeks ago, Karnataka issued show-cause notices to 394 officials across 311 Gram Panchayats over delayed wages.*"
2. **[00:20 – 00:50] Beat 2: Why It Persists (30s)**
   - Explain the paper-vs-database gap: "*The worker holds paper. The state holds a database. Nobody compares them because manual checks take hours, and individual delay fines of ₹21.60 are too small*

to hire a lawyer for—leading to ₹399M in unacknowledged liabilities."

3. **[00:50 – 01:10] Beat 3: The Principle (20s)**
   - *"We built Saakshi on one rule: The AI reads, the code computes, the human decides. Gemma 4 reads the handwriting. It calculates nothing. All math is pure Python. And zero output is generated without human verification."*
4. **[01:10 – 01:40] Beat 4A: Individual Worker Demo (30s)**
   - Open `/audit/new`. Show photo crop beside the input box. Deliberately correct a smudged numeral. Click "Run Audit". Show `/audit/w1` with ₹1,821.60 hero figure and worked arithmetic line. Click "Read Aloud" to hear the speech in Marathi.
5. **[01:40 – 02:25] Beat 4B: The Climax — Muster Reconciler (45s)**
   - Navigate to `/reconcile/borgaon`.
   - Point out the coverage notice: *"Based on 1 of 14 rolls."*
   - Show the 20-worker cross-check grid: 2 unverified names with no paper card in the village, 4 inflated days under Mate Dnyandev Shinde, and complete co-worker mismatch.
   - Show the printable one-page Gram Sabha sheet. *"Across one muster roll, it is ₹6,600 an official must answer for at tomorrow's Gram Sabha."*
6. **[02:25 – 02:45] Beat 5: Proof & Calibration (20s)**
   - Show the benchmark table: *"Tesseract scored 0% on handwriting; Gemma 4 scored 96.6%. And we calibrated confidence: above 0.85, it is 97.4% accurate. Below that, we abstain and ask."*
7. **[02:45 – 03:00] Beat 6: Open Weights & Closing (15s)**
   - Flip the topbar toggle to `[LOCAL OLLAMA]`. Show the trace panel logging local inference.
   - *"The weights are Apache 2.0. State audit units can run this air-gapped on their own laptops. Saakshi: proof for the people who do the work."*

9.4 Common Judge Questions & Winning Answers

Q1: "Why did you use Gemma 4 instead of Gemini 1.5 Pro or GPT-4o?"
Answer:
"Three reasons:

1. **Model Governance & Integrity:** This is a Gemma challenge entry. We hard-enforced `gemma-4-26b-a4b-it` and `gemma-4-31b-it` in `client.py` with zero fallbacks.
2. **Cost-Free Scalability:** Gemma 4 is free on Google AI Studio ($0 in / $0 out). A rural welfare audit system cannot depend on metered API bills that lapse when funding runs out.
3. **Data Sovereignty & Open Weights:** State Social Audit Units handling sensitive citizen records cannot send data to foreign commercial cloud APIs. Gemma 4's Apache 2.0 weights allow identical code to run 100% offline via local Ollama."

Q2: "What if Gemma 4 hallucinates an incorrect wage amount?"

Answer:

"Gemma 4 is physically incapable of hallucinating a wage amount in Saakshi because **it is never allowed to calculate one**. Gemma 4 acts solely as an optical sensor reading raw text into structured schemas. All wage math, statutory daily rate floors (₹300/day), date intervals, and delay compensation are computed by deterministic Python functions in `backend/audit/engine.py`, verified by 21 unit tests. Furthermore, no math runs until a human confirms the cropped field on screen."

Q3: "How do you know model confidence isn't self-reported fiction?"

Answer:

"We answered that objection through empirical calibration in `reference/CALIBRATION.md`. We evaluated 193 fields against ground truth and computed an Expected Calibration Error of 9.18%. Above our calibrated threshold of 0.85, the model was accurate 97.4% of the time. Below 0.85, the system abstains from assumption and prompts human review. In addition, independent deterministic arithmetic validators check the formulas regardless of model confidence."

Q4: "Can this evidence be used in an Indian court?"

Answer:

"Under Section 63 of the Bharatiya Sakshya Adhiniyam, 2023, electronic evidence requires a dual-hash certificate signed by the device custodian (Part A) and an electronic evidence expert (Part B). Saakshi pre-fills Part A with camera-capture and model SHA-256 hashes, verified by our CLI tool. We explicitly label it: '*Court-ready preparation — Part B expert certification still required*'. We provide the complete chain of custody for a social audit advocate to present formally."

End of Master Guide. Saakshi: The AI reads. The code computes. The human decides.